Typisierung

Coders.Eleven – PYTHON ADVANCED

Coders.Eleven PYTHON ADVANCED
Typisierung

#1 Typing Systems

Representation

Stark vs. Schwach

Dynamisch vs. Statisch

Duck Typing

#2 Kopieren

len(x) vs. x.len()

Identity, Equality & Aliasing

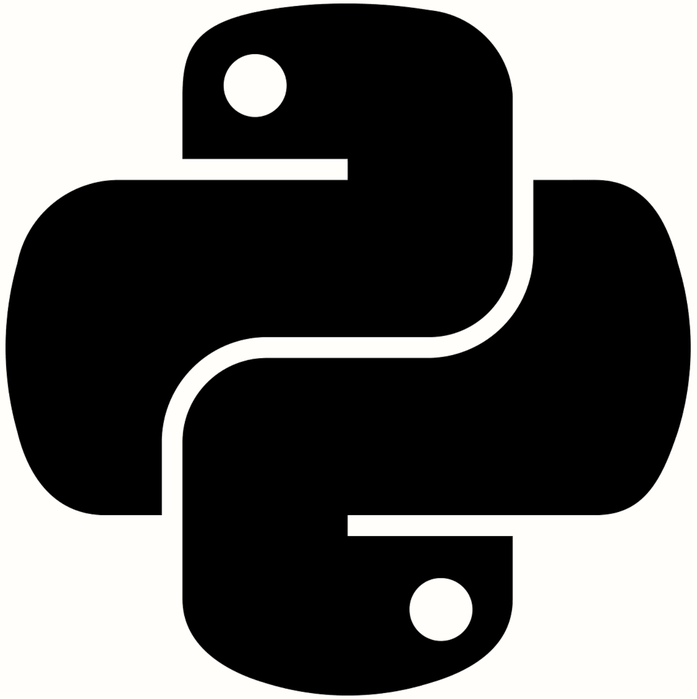
Kopien

#3 Referenzparameter

Call by Sharing

Mutable Types as Parameter Defaults

Defensive Programming



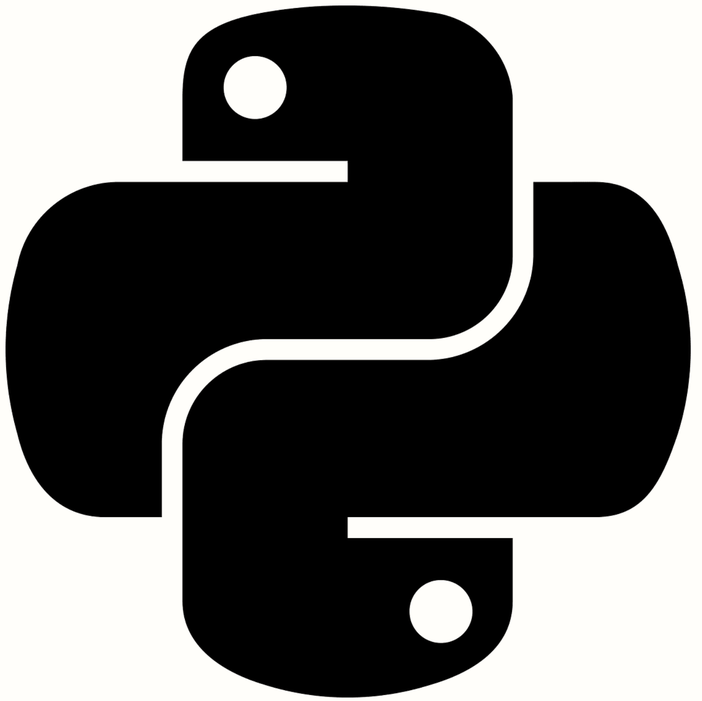
#1 Typing Systems

Representation

Stark vs. Schwach

Dynamisch vs. Statisch

Duck Typing



Python Types

- Stellen Sie sich vor, Sie müssen jemanden erklären, was ein Typ ist, Sie dürfen jedoch nicht die Wörter string, text, boolean, numbers, etc. verwenden
- Was sind die Vorteile davon, insbesondere in Python, wo man keine Typen explizit deklarieren muss?
- Einfache Erklärung: Types sind Kommunikationsmethoden, sie übermitteln Informationen, sie bieten eine Darstellung, über die Nutzer und Computer nachdenken können
  - Darstellung wird unterschieden
    - Mechanical representation
      - Typen vermitteln der Python language selbst Verhaltensweisen und Einschränkungen
    - Semantic representation
      - Typen vermitteln anderen Entwicklern Verhaltensweisen und Einschränkungen

Mechanical representation

- Prozessor „spricht“ kein Python, sondern arbeitet binär
  - Er erkennt nur das Vorhandensein oder Fehlen von Strom in seinen Schaltkreisen
  - Selbiges gilt für den Speicher
- Angenommen unser Speicher sieht so aus:

```
00110010100010010001010010010001001000010000010101
00101010101010000001111111100100101001111110100100
01001000100101001010111011111011010101010101010101

01010000010000010101010100

10100100100100010101000101001001010101001001001001
00011110101011010110100101011100000000000000000111
```

- Ein Haufen an 0er und 1er, für uns jedenfalls sehr schwierig zu lesen

Mechanical representation

- Schauen wir uns den mittleren Teil etwas genauer an:
- Angenommen unser Speicher sieht so aus:

```
01010000 01000001 01010100
```

- Wir können daraus unmöglich erkennen, was diese Zahlen genau bedeuten sollen
- Je nach Computerarchitektur könnte es zB 5259604 oder 5521744 sein bzw. „PAT“
- Aus diesem Grund werden Types benötigt

Beispiel

- Hex-Zeichenfolgen zeigen den Inhalt des Speichers an, der ein Python-Objekt enthält
- Darin enthalten sind Zeiger auf das nächste und vorherige Objekt in einer verketteten Liste (wg. Garbage Collection), Referenzanzahl, Type und die eigentlichen Daten
- Bytes am Ende jedes Werts zeigen den Type (0x544150 oder 0x504154)
- Python weiß also zur Laufzeit genau um welchen Type es sich handelt (wie bei der type()-Funktion)
- Wir wollen robusten Code schreiben, weshalb es wichtig ist zu verstehen, wie Python Typen verwendet

```
from ctypes import string_at
from sys import getsizeof
from binascii import hexlify

a = 0b01010000_01000001_01010100
print(a)
>>> 5259604

# prints out the memory of the variable
print(hexlify(string_at(id(a), getsizeof(a))))
>>> b'0100000000000000607c0549955500000100000000000000054415000'

text = "PAT"
print(hexlify(string_at(id(text), getsizeof(text))))
```

Semantic Representation

- Typen vermitteln Informationen über Zeit und Raum an einen zukünftigen Entwickler
  - Semantische Darstellung ist also ein Kommunikationsmittel
- Typen sagen dem Entwickler welche Verhaltensweisen er erwarten kann
  - Verhaltensweisen sind Operationen, die mit diesem Type einhergehen
  - Sie legen Grenzen, Einschränkungen und Freiheiten fest
- Richtig eingesetzte Typen sind leicht verständlich, schlecht eingesetzte Typen führen zu Hindernissen
- Semantik beschreibt die Bedeutung einer Operation.
  - str(int) und str(datetime.datetime.now()) geben unterschiedlich formatierte Zeichenfolgen zurück
  - Die Bedeutung ist jedoch dieselbe: Ich erstelle aus einem Wert eine Zeichenfolge

Vergleich int vs. datetime

- Kann aus int, float, str erzeugt werden
- Mathematische Operationen
- Relationale Vergleiche
- Bitweise Operatoren
- Konvertierbar mithilfe von str oder repr
- Runden mit ceil, floor oder round
- Mit help(int) kann eine vollständige Liste angezeigt werden

- Kann aus str oder einem int (day/month/year/etc.) erzeugt werden
- Mathematische Operationen (+, -) von time deltas
- Relationale Vergleiche
- Keine bitweisen Operatoren
- Konvertierbar mithilfe von str oder repr
- Runden nicht möglich

- Datetime ist prinzipiell ein Integer, hat jedoch nicht alle Operationen
  - Spezifischer als eine einfache Zahl
  - Entscheidet man sich für spezifische Typen, legt man auch für zukünftige Developer Operationen fest die möglich bzw. nicht möglich sind

Robust Code

- Angenommen Sie kommen in ein bestehendes Projekt mit einer großen Codebase
- Sie finden dort folgende Methode:

```
def close_kitchen_if_past_cutoff_time(point_in_time):
    if point_in_time >= closing_time():
        close_kitchen()
        log_time_closed(point_in_time)
```

- Sie müssen mit dem Parameter arbeiten, wissen aber nicht um welchen Type es sich handelt
  - Ist es ein str, int, datetime oder eine eigene Klasse? Welche Operationen dürfen wir anwenden?
  - Wenn wir die Methode aufrufen wollen bestehen dieselben Fragen
- Treffen wir hier falsche Annahmen und der Code geht in Produktion, ist er weniger robust
- Fix ist: Ein Bug lauert im Code und die Wartbarkeit wird beeinträchtigt
- Wie können wir das verhindern?
  - Wir suchen nach Tests, Dokumentationen oder aufrufenden Code

Robust Code

- Den gesamten Code zu durchforsten, kostet Zeit
- Achtung vor dem „death by a thousand cuts“
- Ursache liegt an der unklaren semantischen Darstellung des Parameters
- Versuchen Sie beim Schreiben von Code die Semantik auch mitzugeben
  - Beispielsweise über einen Kommentar
  - Oder besser: Über Type-Annotations (ab Python 3.5)

```
def close_kitchen_if_past_cutoff_time(point_in_time:
datetime.datetime):
    if point_in_time >= closing_time():
        close_kitchen()
        log_time_closed(point_in_time)
```

  - Nun weiß jeder Developer was von point_in_time erwartet wird, ohne danach zu suchen

Typing Systems

- Type systems bieten Entwicklern die Möglichkeit, Verhaltensweisen und Einschränkungen der Sprache zu modellieren
- Programmiersprachen legen fest, wie ihre Type systems funktionieren (sowohl zur Laufzeit als auch während des Programmierens)

Stark vs. Schwach

- Type systems werden von schwach bis stark eingeteilt
- Programmiersprachen am stärkeren Ende beschränken die Verwendung von Operationen auf die Typen, die sie auch unterstützen
  - Verstößt man also gegen die semantische Darstellung, wird man durch einen Compiler- oder Laufzeitfehler darauf aufmerksam gemacht
  - Beispiele für stark typisierte Sprachen: Haskell, TypeScript, Rust
- Programmiersprachen am schwächeren Ende schränken die Verwendung von Operationen nicht auf die Typen ein, die sie unterstützen
  - Typen werden oft in andere Typen umgewandelt, damit eine Operation Sinn ergibt
  - Vorteil dabei ist die schnellere Geschwindigkeit
  - Beispiele für schwach typisierte Sprachen: JavaScript, Perl
- Python, Java und C# sind eher auf der stärkeren Seite zu finden

Vergleich

- Es gibt nur wenige implizite Typumwandlungen
- Beispiel in Python:

```
>>>[] + {}
TypeError: can only concatenate list (not "dict") to list

>>> {} + []
TypeError: unsupported operand type(s) for +: 'dict' and list
```

- Beispiel in JavaScript:

```
>>> [] + {}
"[object Object]"

>>> {} + []
0
```

- Bzgl. Robustheit sind stark typisierte Programmiersprachen eine große Hilfe, da Fehler eindeutig angezeigt werden (auch wenn erst zur Laufzeit)
- Zeitaufwand zur Fehlerbehebung wird dadurch reduziert

Dynamisch vs. Statisch

- Eine andere Einteilung der Type systems gilt für die mechanische Darstellung
- Sprachen mit einer statischen Typisierung betten die Typeinformationen bereits zur Kompilierungszeit in die Variablen ein
  - Variablen ändern ihren Typ während der Laufzeit also nicht
  - Vorteil davon ist, dass von Beginn weg sicherer Code geschrieben wird
  - Java und C# sind statisch typisiert
- Sprachen mit einer dynamischen Typisierung verknüpfen den Type mit dem Wert bzw. der Variable selbst
  - Vorteil davon ist die hohe Flexibilität sowie die Geschwindigkeit zum Entwickeln (weniger Compilerprobleme)
  - Python ist dynamisch typisiert

Robust Code

- Dynamische Typisierung führt leider dazu, dass es in einigen Fällen schwieriger ist robusten Code zu schreiben
- Da sich der Type zur Laufzeit ändern kann, kann man keine festen Annahmen über eine Variable für ihre gesamte Lebensdauer treffen
- Leider helfen uns dabei auch die Type-Annotations nicht
  - Keine Errors, Warnings, usw.
- Trotzdem gibt es einige Möglichkeiten robusten Code zu schreiben

```
>>> a: int = 5
>>> a = "string"
>>> a
"string"
```

```
>>> a = 5
>>> a = "string"
>>> a
"string"
```

Duck Typing

- Ein ungeschriebenes Gesetz besagt, dass sobald jemand „duck typing“ anspricht, ein anderer mit „If it walks like a duck and it quacks like a duck, then it must be a duck.“ antworten muss
- Problem an dieser Phrase: Es erklärt nicht was duck typing eigentlich ist.
- Duck Typing ist die Fähigkeit, Objekte und Entities in einer Programmiersprache zu verwenden, solange sie einem bestimmten Interface entsprechen.

```
from typing import Iterable
def print_items(items: Iterable):
    for item in items:
        print(item)

print_items([1,2,3])
print_items({4, 5, 6})
print_items({"A": 1, "B": 2, "C": 3})
```

- Bei allen drei Aufrufen iterieren wir durch die Collection und geben jedes Element aus

Duck Typing

- Die Methode weiß erst zur Laufzeit welchen Type der Parameter hat
- Es wird nicht jedes Argument durchsucht und je nach Type etwas anderes gemacht
- print_items prüft nur, ob das übergebene Objekt iteriert werden kann (durch Aufruf einer __iter__ Methode)
- Duck Typing macht das möglich: Solange ein Type die von einer Funktion verwendeten Variablen und Methoden unterstützt, kann dieser Type verwendet werden
- Beispiel für den Fehlerfall:

```
>>> print_items(5)

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in print_items
TypeError: 'int' object is not iterable

>>> '__iter__' in dir(int)
False
>>> '__iter__' in dir(list)
True
```

Duck Typing

- Hier sehen wir, dass es egal ist, ob (in diesem speziellen Beispiel) ein integer oder string übergeben wird, denn beide unterstützen den + Operator
  - Beispielsweise kann auch eine Liste übergeben werden

```
>>>def double_value(value):
>>>    return value + value

>>>double_value(5)
10

>>>double_value("abc")
"abcabc"
```

```
>>>double_value([1, 2, 3])
[1, 2, 3, 1, 2, 3]
```

Robust Code

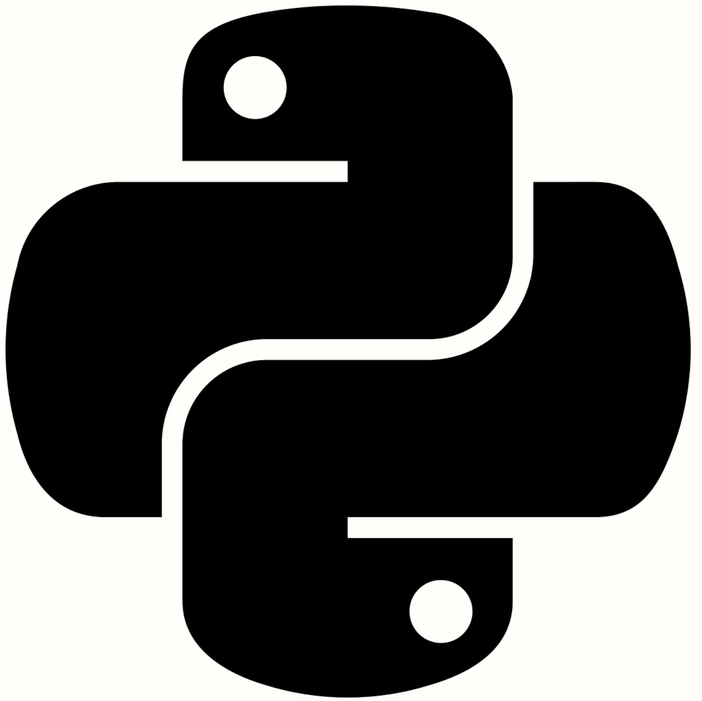
- Duck Typing kann die Robustheit erhöhen, da es die Kombinierbarkeit verbessert
  - Zulassen einer Vielzahl an Typen kann den Bedarf an komplexen Sonderfällen verringern
- Durch übertriebenen Einsatz können jedoch Annahmen untergraben werden, auf die sich ein Developer verlassen kann
  - Bei Codeveränderungen reicht es nicht aus nur die eine Methode zu verändern, sondern der gesamte aufrufende Code muss überprüft werden
- Aus diesen Gründen eignet sich folgendes (leider nicht so leicht von der Zunge gehendes) Sprichwort:
  - „If it walks like a duck and quacks like a duck, and you are looking for things that walk and quack like ducks, then you can treat it as if it were a duck“

#2 Kopieren

len(x) vs. x.len()

Identity, Equality & Aliasing

Kopien



Warum len(x) und nicht x.len()?

- Erinnern Sie sich an „The Zen of Python“: „practicality beats purity“
- Wenn x ein built-in-Object ist, wird keine Methode aufgerufen, da len als Feld gelesen wird
  - Die Anzahl an Elementen einer Collection ist eine gängige Operation und muss daher für grundlegende und vielfältige Typen wie str, list, usw. effizient funktionieren
- Aufgrund der Special-Method __len__, funktioniert len aber auch auf eigenen Objekten
- Dadurch hat meinen einen Kompromiss zwischen der Notwendigkeit effizienter integrierter Objekte und der Konsistenz der Programmiersprache gefunden
- Übrigens ebenfalls aus „The Zen of Python“: „Special cases aren‘t special enough to break the rules.“

Identity, Equality & Aliasing

```
>>> charles = {'name': 'Charles L. Dodgson', 'born': 1832}
>>> lewis = charles  ❶
>>> lewis is charles
True
>>> id(charles), id(lewis)  ❷
(4300473992, 4300473992)
>>> lewis['balance'] = 950  ❸
>>> charles
{'name': 'Charles L. Dodgson', 'born': 1832, 'balance': 950}
```

❶ lewis is an alias for charles.

❷ The is operator and the id function confirm it.

❸ Adding an item to lewis is the same as adding an item to charles.

- Aliasing: lewis und charles sind aliases, da die beiden Variablen an dasselbe Objekt gebunden sind
- Alex ist kein alias für charles, da sie an unterschiedliche Objekte gebunden sind
- Die Objekte, die an alex und charles gebunden sind haben denselben Wert (== Vergleich), aber unterschiedliche Identitäten (is Vergleich)
- id()-Funktion: Liefert die Speicheradresse des Objekts (integer)

Identity, Equality & Aliasing

```
>>> alex = {'name': 'Charles L. Dodgson', 'born': 1832, 'balance': 950}  ❶
>>> alex == charles  ❷
True
>>> alex is not charles  ❸
True
```

❶ alex refers to an object that is a replica of the object assigned to charles.

❷ The objects compare equal because of the __eq__ implementation in the dict class.

❸ But they are distinct objects. This is the Pythonic way of writing the negative identity comparison: a is not b.

- Einziger gebräuchlicher Anwendungsfall von „is“ ist der Vergleich mit None, sonst meistens ==

#2 Kopieren

Relative Immutability of Tuples

- Tupel sind wie die meisten Python-Collections (lists, dicts, sets, etc.) Container, d.h. sie enthalten Objektreferenzen
- Sind die referenzierten Elemente veränderbar, können sie sich ändern, auch wenn sich das Tupel selbst nicht ändert, d.h. die Unveränderlichkeit von Tupeln bezieht sich tatsächlich auf den physischen Inhalt der Tupel-Datenstruktur (also auf die darin enthaltenen Referenzen) und nicht auf die referenzierten Objekte
- Identity vs. Equality wird spannend, wenn ein Objekt kopiert wird, aber selbst andere Objekte beinhaltet
  - Sollen die enthaltenen Objekte dann auch kopiert werden, oder sollen sie gemeinsam genutzt werden?

Beispiel

```
>>> t1 = (1, 2, [30, 40])  ❶
>>> t2 = (1, 2, [30, 40])  ❷
>>> t1 == t2  ❸
True
>>> id(t1[-1])  ❹
4302515784
>>> t1[-1].append(99)  ❺
>>> t1
(1, 2, [30, 40, 99])
>>> id(t1[-1])  ❻
4302515784
>>> t1 == t2  ❼
False
```

❶ `t1` is immutable, but `t1[-1]` is mutable.

❷ Build a tuple `t2` whose items are equal to those of `t1`.

❸ Although distinct objects, `t1` and `t2` compare equal, as expected.

❹ Inspect the identity of the list at `t1[-1]`.

❺ Modify the `t1[-1]` list in place.

❻ The identity of `t1[-1]` has not changed, only its value.

❼ `t1` and `t2` are now different.

Shallow Copies

- Die einfachste Art Listen (bzw. die meisten built-in mutable collections) zu kopieren, ist über den Konstruktor bzw. über list[:] (mehr dazu in der nächsten Einheit)
- Standardmäßig wird eine shallow copy erzeugt, d.h. die äußere Collection wird kopiert, aber mit Referenzen auf dieselben Elemente gefüllt, die in der ursprünglichen Collection waren
  - Spart Speicherplatz und ist auch kein Problem, wenn alle Elemente immutable sind
  - Sind Elemente mutable, können Probleme auftreten

```
l1 = [3, [66, 55, 44], (7, 8, 9)]
l2 = list(l1)          ❶
l1.append(100)         ❷
l1[1].remove(55)       ❸
print('l1:', l1)
print('l2:', l2)
l2[1] += [33, 22]      ❹
l2[2] += (10, 11)      ❺
print('l1:', l1)
print('l2:', l2)
```

❶ l2 is a shallow copy of l1. This state is depicted in Figure 6-3.

❷ Appending 100 to l1 has no effect on l2.

❸ Here we remove 55 from the inner list l1[1]. This affects l2 because l2[1] is bound to the same list as l1[1].

❹ For a mutable object like the list referred by l2[1], the operator += changes the list in place. This change is visible at l1[1], which is an alias for l2[1].

❺ += on a tuple creates a new tuple and rebinds the variable l2[2] here. This is the same as doing l2[2] = l2[2] + (10, 11). Now the tuples in the last position of l1 and l2 are no longer the same object. See Figure 6-4.

Shallow Copies

```
l1: [3, [66, 44], (7, 8, 9), 100]
l2: [3, [66, 44], (7, 8, 9)]
l1: [3, [66, 44, 33, 22], (7, 8, 9), 100]
l2: [3, [66, 44, 33, 22], (7, 8, 9, 10, 11)]
```

- Shallow Copies sind einfach zu erstellen, aber mit Vorsicht zu genießen

Deep and Shallow Copies

- Deep copies haben keine gemeinsamen Referenzen auf eingebettete Objekte
- Im Modul copy gibt es die Funktionen deepcopy() und copy(), um deep- und shallow-copies zu erzeugen

```
class Bus:

    def __init__(self, passengers=None):
        if passengers is None:
            self.passengers = []
        else:
            self.passengers = list(passengers)

    def pick(self, name):
        self.passengers.append(name)

    def drop(self, name):
        self.passengers.remove(name)
```

Beispiel

```
>>> import copy
>>> bus1 = Bus(['Alice', 'Bill', 'Claire', 'David'])
>>> bus2 = copy.copy(bus1)
>>> bus3 = copy.deepcopy(bus1)
>>> id(bus1), id(bus2), id(bus3)
(4301498296, 4301499416, 4301499752)  ❶
>>> bus1.drop('Bill')
>>> bus2.passengers
['Alice', 'Claire', 'David']          ❷
>>> id(bus1.passengers), id(bus2.passengers), id(bus3.passengers)
(4302658568, 4302658568, 4302657800)  ❸
>>> bus3.passengers
['Alice', 'Bill', 'Claire', 'David']  ❹
```

❶ Using `copy` and `deepcopy`, we create three distinct `Bus` instances.

❷ After `bus1` drops `'Bill'`, he is also missing from `bus2`.

❸ Inspection of the `passengers` attributes shows that `bus1` and `bus2` share the same list object, because `bus2` is a shallow copy of `bus1`.

❹ `bus3` is a deep copy of `bus1`, so its `passengers` attribute refers to another list.

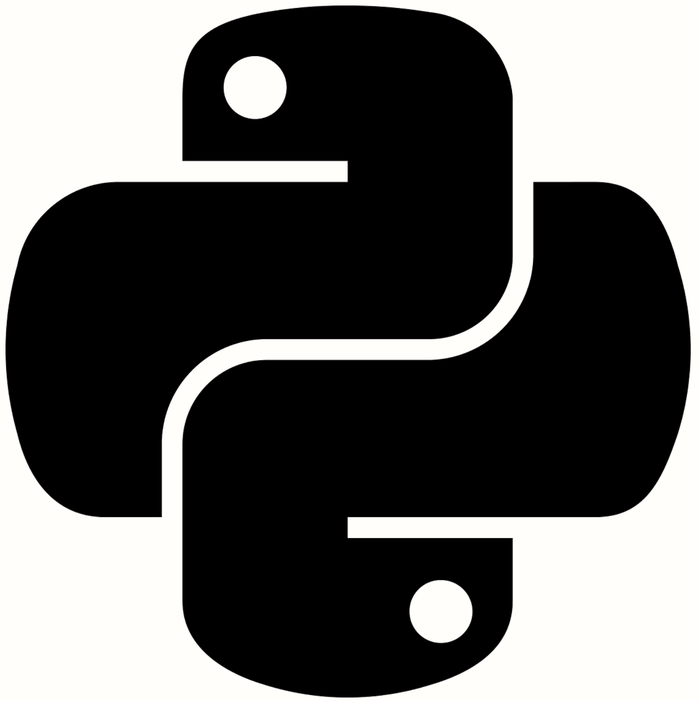
- Achtung: deep-copies können zu Endlosschleifen führen, wenn Objekte sich gegenseitig referenzieren
- Deep-copies können auch zu „deep“ werden, zB wenn singeltons kopiert werden

#3
Referenzparameter

Call by Sharing

Mutable Types as Parameter Defaults

Defensive Programming



Call by Sharing

- In Python können Parameter nur mittels „Call by Sharing“ übergeben werden
  - Wie auch in JavaScript, Ruby, Java (Referenztypen, Werttypen verwenden „Call by Value“)
- Jeder formale Parameter der Funktion erhält eine Kopie jeder Referenz in den Argumenten
  - D.h. Parameter innerhalb der Funktion werden zu Aliasen der tatsächlichen Argumente
- Dadurch kann die Funktion jedes als Parameter übergebene mutable Objekt ändern (aber nicht die Identität)

```
>>> def f(a, b):
...     a += b
...     return a
...
>>> x = 1
>>> y = 2
>>> f(x, y)
3
>>> x, y  ❶
(1, 2)
```

```
>>> a = [1, 2]
>>> b = [3, 4]
>>> f(a, b)
[1, 2, 3, 4]
>>> a, b  ❷
([1, 2, 3, 4], [3, 4])
>>> t = (10, 20)
```

```
>>> u = (30, 40)
>>> f(t, u)  ❸
(10, 20, 30, 40)
>>> t, u
((10, 20), (30, 40))
```

❶ The number x is unchanged.

❷ The list a is changed.

❸ The tuple t is unchanged.

Mutable Types as Parameter Defaults

- Optionale Parameter mit Defaultwerten sind praktisch, insbesondere in der Weiterentwicklung von APIs, da so die Abwärtskompatibilität gewahrt werden kann
- Trotzdem sollten mutable Objekte als Parameter Defaults vermieden werden
- Im folgenden Beispiel haben wir uns im Konstruktor ein if gespart, da wir einen Default Parameter verwenden

```
class HauntedBus:
    """A bus model haunted by ghost passengers"""

    def __init__(self, passengers=[]):  ❶
        self.passengers = passengers  ❷

    def pick(self, name):
        self.passengers.append(name)  ❸

    def drop(self, name):
        self.passengers.remove(name)
```

❶ When the `passengers` argument is not passed, this parameter is bound to the default list object, which is initially empty.

❷ This assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the default list, when no `passengers` argument is given.

❸ When the methods `.remove()` and `.append()` are used with `self.passengers`, we are actually mutating the default list, which is an attribute of the function object.

Beispiel

```
>>> bus1 = HauntedBus(['Alice', 'Bill'])  ❶
>>> bus1.passengers
['Alice', 'Bill']
>>> bus1.pick('Charlie')
>>> bus1.drop('Alice')
>>> bus1.passengers  ❷
['Bill', 'Charlie']
>>> bus2 = HauntedBus()  ❸
>>> bus2.pick('Carrie')
>>> bus2.passengers
['Carrie']
>>> bus3 = HauntedBus()  ❹
>>> bus3.passengers  ❺
['Carrie']
>>> bus3.pick('Dave')
>>> bus2.passengers  ❻
['Carrie', 'Dave']
>>> bus2.passengers is bus3.passengers  ❼
True
>>> bus1.passengers  ❽
['Bill', 'Charlie']
```

❶ `bus1` starts with a two-passenger list.

❷ So far, so good: no surprises with `bus1`.

❸ `bus2` starts empty, so the default empty list is assigned to `self.passengers`.

❹ `bus3` also starts empty, again the default list is assigned.

❺ The default is no longer empty!

❻ Now `Dave`, picked by `bus3`, appears in `bus2`.

❼ The problem: `bus2.passengers` and `bus3.passengers` refer to the same list.

❽ But `bus1.passengers` is a distinct list.

- Darum wird „None“ gerne als Standardwert für mutable Werte verwendet

Defensive Programming

- Hat man einen mutable Parameter, muss man darüber nachdenken, ob der Aufrufer erwartet, dass das übergebene Argument verändert wird
- Die Herausforderung liegt darin, den Erwartungen der Developer mit denen des Aufrufenden abzustimmen
- Beispiel aus Client-Sicht:

```
>>> basketball_team = ['Sue', 'Tina', 'Maya', 'Diana', 'Pat']  ❶
>>> bus = TwilightBus(basketball_team)  ❷
>>> bus.drop('Tina')  ❸
>>> bus.drop('Pat')
>>> basketball_team  ❹
['Sue', 'Maya', 'Diana']
```

❶ `basketball_team` holds five student names.

❷ A `TwilightBus` is loaded with the team.

❸ The `bus` drops one student, then another.

❹ The dropped passengers vanished from the basketball team!

Beispiel

```
class TwilightBus:
    """A bus model that makes passengers vanish"""

    def __init__(self, passengers=None):
        if passengers is None:
            self.passengers = []  ❶
        else:
            self.passengers = passengers  ❷

    def pick(self, name):
        self.passengers.append(name)

    def drop(self, name):
        self.passengers.remove(name)  ❸
```

❶ Here we are careful to create a new empty list when `passengers` is `None`.

❷ However, this assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the actual argument passed to `__init__` (i.e., `basketball_team` in Example 6-14).

❸ When the methods `.remove()` and `.append()` are used with `self.passengers`, we are actually mutating the original list received as an argument to the constructor.

- Problem: Die übergebene Liste im Konstruktor wird „aliasiert“, stattdessen sollte er eine eigene Liste führen
- Fix: Im Konstruktor sollte, wenn der Parameter übergeben wird, self.passengers mit einer Kopie initialisiert werden

Beispiel

```
def __init__(self, passengers=None):
    if passengers is None:
        self.passengers = []
    else:
        self.passengers = list(passengers) ❶
```

❶ Make a copy of the `passengers` list, or convert it to a `list` if it's not one.

- Dadurch, dass wir uns nun intern um die Liste kümmern, haben wir keinen Einfluss mehr auf das Argument
- Zusätzlich ist die Lösung flexibler, da nun jedes iterierbare Objekt übergeben werden kann, da der Listenkonstruktor jedes interierbare Objekt akzeptiert

Literaturverzeichnis

- Robust Python – Write Clean and Maintainable Code, 2nd Edition, Patrick Viafore
- Fluent Python – Clear, Concise, and Effective Programming, 2nd Edition, Luciano Ramalho